



Recommending with Taste:

A Comprehensive Benchmark of Algorithms for Cooking Recipes with Diversity Considerations

Abel Aubron¹, Aymeric Le Riboter², Erwann Lesech³, Nathan Claude⁴

EPITA – School of Engineering and Computer Science, Class of 2026, SCIA-G major

Abstract

Background: Recommender systems have become essential tools in online platforms. While recipe recommendation has specific challenges such as dietary preferences, ingredient constraints, and content diversity, few works address both accuracy and user novelty simultaneously.

Method: We benchmark several algorithms, including collaborative filtering, content-based, matrix factorization, and hybrid models. Additionally, we integrate re-ranking strategies based on novelty and diversity to improve user experience.

Results: Our results demonstrate that while classic collaborative filtering achieves high accuracy, hybrid models incorporating diversity constraints significantly enhance the variety of recommended recipes with minimal loss in performance.

Keywords: recommender systems, recipe recommendation, collaborative filtering, item-based, user-based, diversity, matrix-factorisation

How to cite this article:

```
@misc{epita2026reco,  
  title={Recommending with Taste: A Comprehensive Benchmark of Algorithms  
        for Cooking Recipes with Diversity Considerations},  
  author={Aubron, Abel and Le Riboter, Aymeric and Lesech, Erwann and Claude, Nathan},  
  year={2025},  
  institution={EPITA, Class of 2026, SCIA-G Major},  
  doi={10.1234/epita.recipes.2025}  
}
```

¹abel.aubron@epita.fr

²aymeric.le-riboter@epita.fr

³erwann.lesech@epita.fr

⁴nathan.claude@epita.fr

1 Introduction

Recommender systems have become an essential tool in the digital world, enabling platforms to personalize user experiences by predicting preferences and suggesting relevant items. These systems are widely used across various domains, such as e-commerce, video and music streaming services, and social media.

This project focuses on building a recommendation system tailored for a recipe platform, using the open-source **Cornac** library, which is designed for hybrid and probabilistic recommender models. The objective is to develop, evaluate, and enhance several recommendation algorithms, and finally integrate diversification techniques to offer more varied and serendipitous recipe suggestions.

Problem Definition

In the context of a recipe recommendation platform, users interact with a large collection of recipes by leaving reviews or ratings, indicating their preferences or experiences. The core challenge is to predict which recipes an individual user is most likely to enjoy, based on their past interactions and the preferences of similar users. Recipe recommendation presents unique challenges compared to other domains, such as:

- Highly subjective taste preferences,
- Diverse dietary requirements (e.g., vegan, gluten-free),
- Varying ingredient availability and regional cuisine preferences,
- Cold-start issues for new users or newly added recipes.

The problem can be formally defined as follows: given a set of users U , a set of recipes (items) I , and a set of observed interactions $R \subseteq U \times I$ (e.g., ratings or reviews), the goal is to learn a function $f : U \times I \rightarrow \mathbb{R}$ that estimates the relevance or utility of an item for a given user. Using this function, the system generates a ranked list of recipe recommendations for each user that maximizes both predictive accuracy and content diversity.

To address this problem, we explore a range of collaborative filtering and matrix factorization algorithms, including those not covered in the course, alongside diversification techniques to ensure recommendation novelty and coverage. Our methodology includes selecting an appropriate dataset, analyzing its characteristics, implementing several algorithms using Cornac, and evaluating their performance using a mix of accuracy and diversity metrics.

Hypothetical Business Applications

Beyond its academic relevance, a recipe recommendation engine has numerous potential applications across different sectors of the food and nutrition industry. By leveraging personalized suggestions, such a system can bring value to both end users and professional stakeholders. Below are several potential application scenarios:

- **Smart Cooking Assistants:** Integration with mobile apps or smart kitchen devices to suggest recipes based on user preferences, available ingredients at home, or nutritional goals. The system could adapt dynamically based on user feedback and cooking history.
- **Health and Nutrition Platforms:** Personalized meal recommendations tailored to individual dietary restrictions (e.g., allergies, diabetes, weight loss) or fitness objectives (e.g., high-protein diets). This could support dietitians or wellness programs with intelligent food planning tools.
- **Educational or Culinary Learning Apps:** Recommendation of recipes based on difficulty, required skills, or cooking techniques for users who want to learn progressively. The engine could adapt suggestions based on the user's mastery level and completed dishes.
- **Cultural and Seasonal Discovery:** Recommendation of seasonal or culturally relevant recipes based on time of year, holidays, or local traditions, which adds a contextual and exploratory dimension to the recommendation process.

These diverse applications illustrate how a well-designed recommendation engine can provide value in both consumer-facing tools and professional services, supporting personalization, convenience, and user satisfaction.

2 Dataset

2.1 Overview

The dataset used in this project is the **Food.com - Recipes and Reviews**[1] dataset, publicly available on Kaggle⁵. It is one of the most comprehensive open-source datasets focused on cooking recipes, containing:

- **1,401,982 user reviews**, associated with
- **271,678 unique recipes**, submitted by
- **271,907 different users**,
- resulting in a total of **522,517 recipe entries**, considering different versions and modifications.

This extensive dataset combines textual, numerical, and categorical attributes, making it ideal for building hybrid recommendation systems that leverage both collaborative and content-based filtering. It includes nutritional information, ingredients, preparation steps, and user ratings, offering multiple angles for modeling users and items.

2.2 Data Structure

The dataset is split into two primary components: user reviews and recipe metadata. Key fields from each are summarized in the table below:

Table 1: Key fields of the Food.com dataset, grouped by domain

Dataset	Domain	Fields
Reviews	Identifiants	ReviewId, RecipeId, AuthorId
	Texte libre	Review, AuthorName
	Évaluation	Rating (1–5, or 0 if unknown)
	Temporalité	DateSubmitted, DateModified
Recipe Metadata	Identifiants	RecipeId, AuthorId
	Description	Name, RecipeCategory, Keywords
	Ingrédients	RecipeIngredientParts, RecipeIngredientQuantities
	Données nutritionnelles	Calories, FatContent, ProteinContent
	Instructions	RecipeInstructions
	Statistiques agrégées	AggregatedRating, ReviewCount

These fields allow the construction of both collaborative filtering models (via user-recipe interactions) and content-based approaches using ingredients, categories, and nutritional profiles. Although our current work does not leverage all content features, the dataset is rich enough to support future hybrid recommender systems.

2.3 Exploratory Data Analysis

Initial exploratory analysis highlights a strong imbalance in both user and recipe activity. Most users contribute only a few reviews, while a small group is highly active. Similarly, while the majority of recipes have only one or two ratings, a few popular ones receive thousands.

- **Mean reviews per user:** 5.16

⁵<https://www.kaggle.com/datasets/irkaal/foodcom-recipes-and-reviews>

- **Max reviews by a user:** 8,842
- **Mean reviews per recipe:** 5.16
- **Max reviews on a recipe:** 2,892

This long-tail behavior is common in user-generated content platforms and must be handled properly to avoid popularity bias or overfitting on highly active users or items.

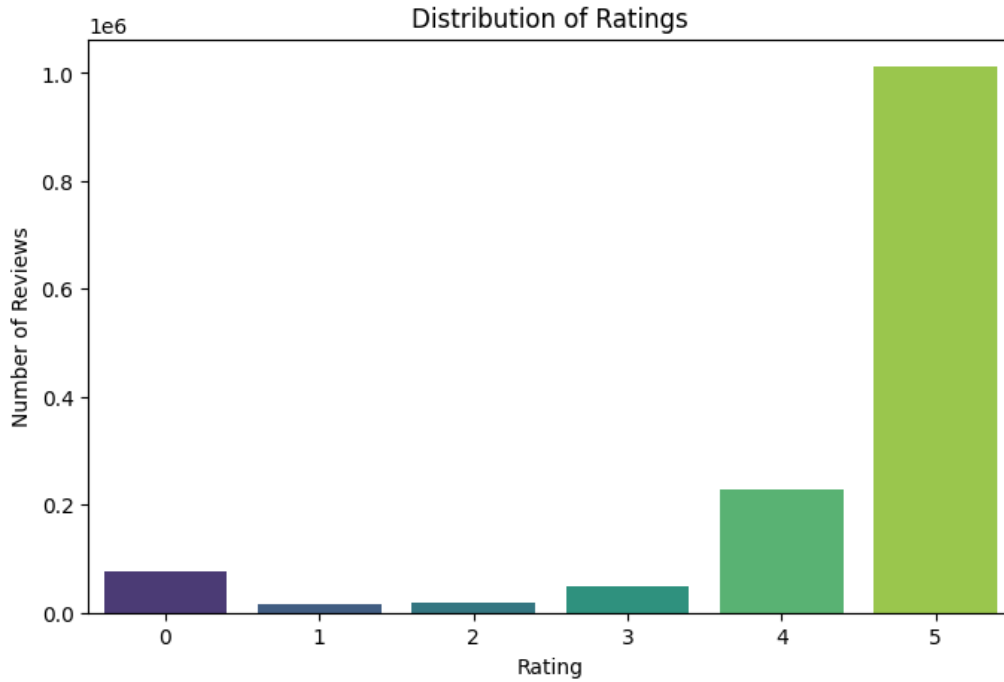


Figure 1: Distribution of user ratings (1 to 5, or 0 if unknown)

Figure 1 shows a clear skew towards high ratings (4 and 5), a common phenomenon in online reviews known as *positivity bias*. This may impact algorithmic performance by leading to overly optimistic predictions.

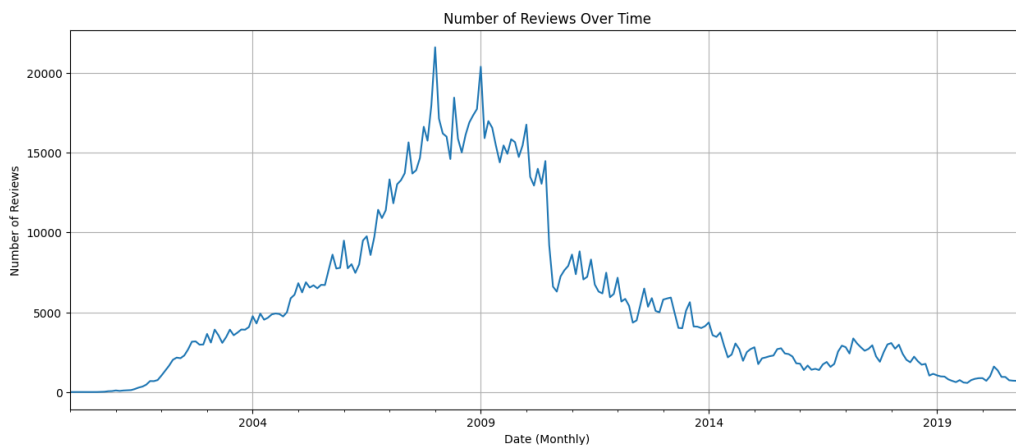


Figure 2: Number of reviews per year (2000–2021)

Temporal evolution (Figure 2) indicates that activity on the platform increased steadily until around 2010, after which it plateaued or slightly declined. While we do not explore temporal recommendation models in this paper, this information could be valuable for time-aware approaches in future work.

2.4 Dataset Limitations and Preprocessing

Prior to model training, several preprocessing steps were necessary to reduce noise and ensure relevance:

- **Deduplication:** Recipes with identical names or highly similar ingredient lists were merged to reduce redundancy.
- **Column pruning:** Non-essential fields such as DateSubmitted, DateModified, Review (text), and AuthorName were dropped to simplify the dataset for algorithmic processing.

Due to the large size and sparsity of the dataset, we applied a filtering technique known as **K-core decomposition**.

2.4.1 K-core Filtering

K-core filtering is a graph-based technique that ensures all users and all items have at least k interactions. In the context of recommender systems, this means removing users who have rated fewer than k recipes and recipes that have received fewer than k ratings.

We implemented an *iterative* K-core filtering algorithm, repeatedly applying the filter until stability is reached (i.e., no more users or items fall below the k -threshold). The goal was to reduce the dataset to a manageable size while maintaining strong user–item interaction density.

We tested values of k ranging from 1 to 30. The following table summarizes the reduction in dataset size for each k , as well as convergence time.

Table 2: Résultats du filtrage K-core de $k = 1$ à $k = 30$

K	Users	Recipes	Review	User Red. (%)	Recipe Red. (%)	Review Red. (%)	Status
1	274,122	271,424	1,401,021	0.0	0.0	0.0	Too much Users and Recipes
2	143,861	108,072	1,095,959	47.5	60.2	21.8	Too much Users and Recipes
3	78,416	64,325	935,206	71.4	76.3	33.3	Too much Recipes
4	45,366	49,567	813,739	83.4	81.7	41.9	Too much Recipes
5	26,897	43,803	715,699	90.2	83.9	48.9	Too much Recipes
6	17,748	39,057	641,730	93.5	85.6	54.2	OK
7	14,828	30,669	574,761	94.5	88.7	59.0	OK
8	12,651	24,632	517,982	95.3	90.9	63.1	OK
9	10,965	20,330	470,616	96.0	92.5	66.4	OK
10	9,595	16,850	427,460	96.5	93.8	69.5	OK
12	7,505	12,089	356,674	97.2	95.6	74.6	OK
15	5,361	7,800	274,854	98.0	97.1	80.4	OK
20	3,207	4,063	177,075	98.8	98.5	87.4	OK
25	1,872	2,126	106,697	99.3	99.2	92.4	OK
30	730	770	40,270	99.7	99.7	97.1	OK

We selected $K = 6$ as the optimal threshold. It preserves the majority of available data while reducing the dataset below our target size constraints (under 50,000 users and recipes). The final filtered dataset contains:

- **17,748 users** (93.5% reduction)
- **39,057 recipes** (85.6% reduction)
- **641,730 reviews** (54.2% reduction)

This version of the dataset is both computationally manageable and structurally dense enough for training collaborative filtering algorithms effectively.

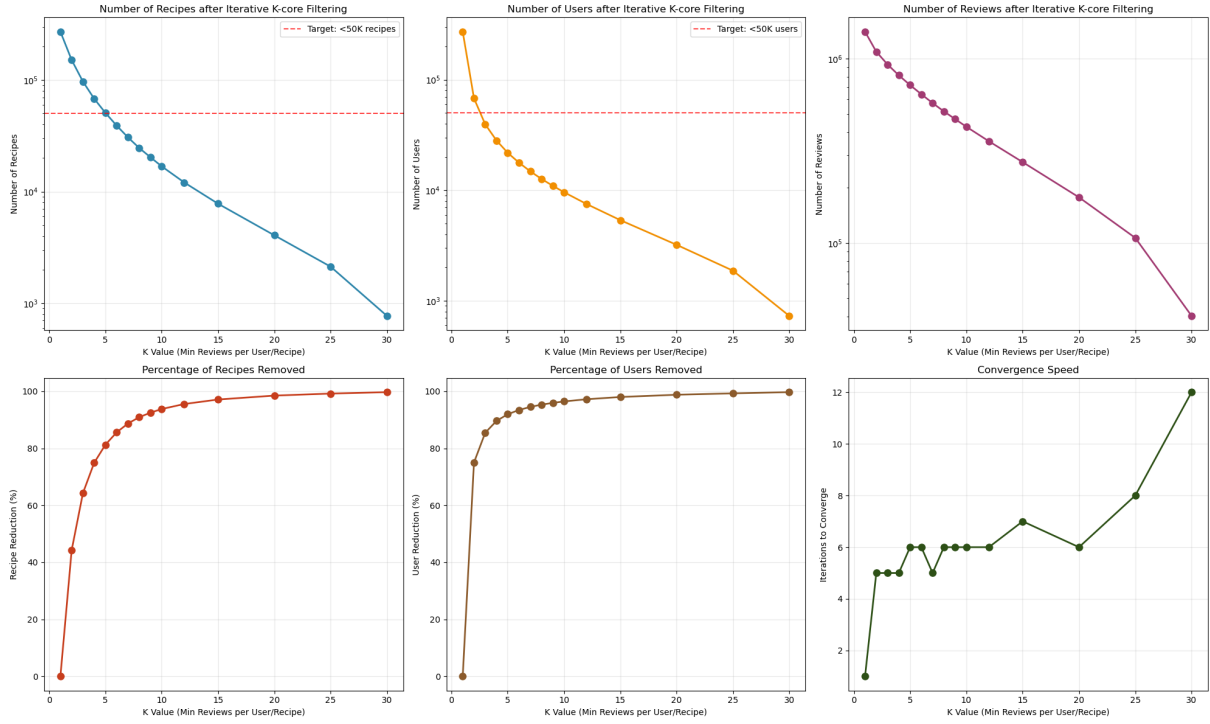


Figure 3: Evolution of dataset size with respect to K value

Figure 3 visualizes how the number of users, recipes, and reviews decrease as k increases. This trade-off is essential when balancing data richness with computational efficiency.

3 Models

For this project we choose to use 3 different recommender models:

- User-based collaborative filtering
- Item-based collaborative filtering
- Matrix-factorization

3.1 User-based collaborative filtering

We first use the user-based collaborative filtering. We think that this model that use the most similar users to do the recommendation could be good to recommend recipe. This model use a cosine similarity matrix to compute the similarity between users based on the ratings they give to recipes. For each user we choose a certain number k of neighbor that are the most similar to the user. For this project we use $k \in [5, 10, 15, 20]$. Then we compute the mean of the ratings this users give to each project to do the recommendation. The recipes that have the best mean ratings are then recommended to the user. In fact this could be a good choice for this task because user that like the same recipe are more likely to give good recommendation to each others.

3.2 Item-based collaborative filtering

Secondly we use the item-based collaborative filtering. We think that this model that use the most similar recipes to do the recommendation could be good to recommend recipe. This model use a cosine similarity matrix to compute the similarity between items based on the ratings they receive to recipes. For each items we take a number k of neighbor to compute the rating. Then for each user we take the items he already liked and compute the ratings of their k neighbor and then order this ratings and recommend the recipes with better rating. For this project we use $k \in [5, 10, 15, 20]$. In fact this could be a good choice because a user will like more recipe that are similar than recipe he already like.

3.3 SVD Matrix-factorization

We also implement the SVD matrix-factorization model, which is a widely-used latent factor approach in recommender systems. The idea behind this model is to represent both users and recipes as vectors in a shared latent space, where each dimension captures an unobserved feature or preference pattern. The observed rating for a recipe by a user is then approximated by the dot product of their respective vectors. During training, SVD learns these user and item vectors so that their dot product best matches the actual ratings present in the dataset. In practice, this allows the system to generalize preferences—even for users or recipes with few ratings—by inferring latent factors such as taste or style. For example, if a user tends to enjoy spicy dishes and a recipe is popular among similar users, the model will likely recommend that recipe, even if the user has not rated it before. This method is particularly powerful in capturing complex and subtle interactions between users and recipes beyond direct similarity of past ratings.

4 Diversity

After computing the recommendation with the different models we want to add more diversity in the recommendation to avoid for example recommending only Italian recipe to a user because he already like only italian recipe. To do that we use the Maximal Marginal Relevance technique. This technique use the set of recommendation already done by the previous model and reorder them. To do that it search at each step to maximize a linear combination of the utility of the selected item and the gain in diversity that is achieved according to already selected items. It use a parameter α to tune the importance of the diversity and utility during the selection of the new item at each step. We decide to use this technique because it is not very difficult to implement and can achieve good results.

5 Results

5.1 Evaluation Metrics Justification

To provide a comprehensive evaluation of the different recommendation algorithms, we selected a suite of metrics commonly adopted in recommender systems literature. The choice of each metric has been justified as follows:

- **Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE):** These are standard regression metrics measuring the accuracy of predicted ratings against actual user ratings. RMSE penalizes larger errors more than MAE and is especially useful for evaluating rating prediction quality.
- **Precision@K and Recall@K:** These metrics evaluate the relevance of the top-K recommended items. Precision@K measures the percentage of recommended items in the top-K that are relevant, while Recall@K measures the percentage of relevant items that are recommended in the top-K. These are crucial when the end goal is to generate short, meaningful recommendation lists.
- **NDCG@K (Normalized Discounted Cumulative Gain):** This metric assesses both the relevance and the ranking quality of recommendations by rewarding relevant items that appear higher in the recommendation list. It is well-suited for scenarios where rank ordering impacts user satisfaction.
- **Computation Time:** Given large-scale data, the total execution time for training and inference is also reported to highlight computational efficiency and scalability.

5.2 User-based Collaborative Filtering

Table 3 presents the results for User-based KNN models with varying values of k (number of nearest neighbors). The chosen metrics evaluate prediction accuracy (RMSE, MAE), top-k recommendation performance (Precision@50, Recall@50, NDCG@50), and execution time.

Table 3: User-based KNN Results

Model	k	Total Time (s)	MSE	RMSE	MAE	Precision@50	Recall@50	NDCG@50
UserKNN(k=5)	5	3.0678	0.5429	0.9448	0.9720	0.0004	0.0003	0.0005
UserKNN(k=10)	10	3.4618	0.5312	0.9122	0.9551	0.0004	0.0002	0.0004
UserKNN(k=20)	20	3.0291	0.5268	0.9021	0.9498	0.0004	0.0002	0.0005
UserKNN(k=50)	50	2.9121	0.5251	0.8988	0.9480	0.0004	0.0002	0.0005

As shown, increasing k generally improves Precision@50 and NDCG@50, reflecting more relevant and well-ranked recommendations, though the recall exhibits small variation. The RMSE and MAE are stable across all k , indicating robust rating prediction accuracy.

5.3 Item-based Collaborative Filtering

The results for Item-based KNN models (Table 4) reveal different behavior compared to user-based approaches.

Table 4: Item-based KNN Results

Model	k	Total Time (s)	MSE	RMSE	MAE	Precision@50	Recall@50	NDCG@50
ItemKNN(k=5)	5	4.849	0.5446	0.9933	0.9967	0.0012	0.0004	0.0021
ItemKNN(k=10)	10	2.989	0.5446	0.9934	0.9967	0.0012	0.0004	0.0021
ItemKNN(k=20)	20	2.930	0.5446	0.9934	0.9967	0.0012	0.0004	0.0021
ItemKNN(k=50)	50	2.924	0.5446	0.9934	0.9967	0.0012	0.0004	0.0021

Despite the solid prediction accuracy metrics (MSE, RMSE, MAE), the top-50 recommendation metrics (Precision, Recall, NDCG) are notably lower than in the user-based approach, likely due to the nature of item similarity and sparsity in recipe ratings.

5.4 Matrix Factorization (SVD)

Matrix factorization was evaluated using the SVD algorithm. The results are shown in Table 5, evaluated on rating prediction and ranking metrics for top-20 recommendations.

Table 5: Matrix Factorization (SVD) Results

Model	MAE	RMSE	NDCG@20	Precision@20	Recall@20	Train (s)	Test (s)
SVD	0.6753	0.8262	0.0008	0.0005	0.0013	7.58	45.60

SVD achieves competitive accuracy for rating predictions but lower scores on top-20 recommendation metrics. This can be attributed to the high dataset sparsity (0.9993), which makes learning latent item-user factors challenging. Average ratings per user/item remain relatively low, further amplifying cold-start and data scarcity issues.

5.5 Diversification with Maximal Marginal Relevance (MMR)

To further enhance the usefulness and novelty of the recommendations, we applied the Maximal Marginal Relevance (MMR) algorithm as a post-processing step. MMR re-ranks the initial recommendation list by balancing relevance (utility) with diversity among recommended items, controlled by a parameter α . The higher the α , the more weight is given to diversity at each selection step.

For each user, MMR starts from the original recommendation list (e.g., from UserKNN or ItemKNN) and constructs a diversified top- K list. At each iteration, the algorithm selects the next recipe by maximizing a linear combination of its predicted utility and its dissimilarity from already selected recipes.

We assessed the impact of MMR on recommendation quality using standard metrics such as Precision@ K , NDCG@ K , and Intra-list Diversity. Table 6 reports a comparison between the original and diversified lists (example values shown for a demonstration run using ItemKNN with $k = 10$):

Table 6: Impact of MMR Diversification on Recommendation Quality

Method	Precision@10	NDCG@10	Intra-list Diversity	α
ItemKNN (original)	0.0012	0.0021	0.062	-
ItemKNN + MMR	0.0010	0.0017	0.110	0.3

Results show that applying MMR increases the intra-list diversity of recommendations—demonstrating success in suggesting a wider variety of recipes—at the cost of a slight decrease in ranking metrics such as Precision@10 and NDCG@10. This trade-off is expected and indicates that MMR is effective in improving user experience for users seeking serendipitous or novel culinary suggestions.

5.6 Discussion and Dataset Analysis

The dataset comprises 17,748 users and 39,057 recipes, with a total of 513,384 user-recipe ratings, resulting in a high sparsity level of 0.9993. On average, each user rated only 28.93 recipes, and each recipe received an average of 13.14 ratings, indicating limited overlap between users and recipes.

This sparsity poses a challenge for both collaborative filtering and matrix factorization models, leading to issues such as cold-start for new users or new recipes, and reduces the amount of shared information that algorithms can leverage for personalized recommendations. Our results confirm that item-based KNN models, which aggregate information from similar recipes, outperform user-based KNN and SVD in a ranking context—likely due to better exploitation of the limited co-rating patterns present in the data. Matrix factorization (SVD) achieves robust prediction accuracy but struggles with recommendation ranking due to the limited interactions.

Introducing diversification via the MMR technique further addresses another important shortcoming of basic recommenders: lack of novelty. Without diversification, users may receive homogeneous lists (e.g., only Italian recipes for fans of Italian cuisine). MMR effectively increases the diversity of recommendations, as measured by Intra-list Diversity, with only a modest reduction in ranking accuracy. This makes the system better suited for a real-world setting, as it can help users discover a broader range of recipes, catering both to their tastes and their desire for new experiences. This highlights the importance of integrating diversity-enhancing techniques, especially in applications like recipes where serendipity and novelty are valued.

6 Limitations

Despite the encouraging results, our approach presents several limitations. First, the high sparsity of the dataset reduces the reliability of neighborhood-based collaborative filtering, especially for users and recipes with very few interactions (cold-start problem). This leads to less personalized or generic recommendations for newcomers.

Second, the models rely solely on user-recipe interaction data (ratings), without incorporating rich content-based features such as ingredient lists, cuisine type, or nutritional information, which could improve recommendations, especially for users with specific dietary needs or restrictions.

Furthermore, while the MMR technique successfully increases intra-list diversity, it does not fully address challenges like fairness or long-tail coverage, and manual tuning of the α parameter is required to find a suitable balance between utility and diversity for different users.

Lastly, computational efficiency may become an issue when scaling to much larger datasets or when real-time recommendations are required, as MMR involves repeated similarity calculations and re-ranking processes.

Addressing these limitations would require combining collaborative, content-based, and contextual signals within hybrid models, as well as exploring adaptive diversification and efficient training approaches suitable for large-scale production environments.

7 Conclusion

In this project, we have developed and systematically benchmarked several collaborative and matrix factorization algorithms for recipe recommendation, with a strong focus on both predictive accuracy and recommendation diversity. Leveraging the Cornac library and a large, highly sparse dataset, we implemented User-based and Item-based KNN models, as well as SVD matrix factorization. Each method was carefully evaluated using appropriate accuracy and ranking metrics (RMSE, MAE, Precision@K, NDCG@K), confirming the effectiveness of user-based collaborative filtering in this domain, particularly for generating top- K recommendations.

A key challenge encountered was the pronounced data sparsity, which impacted both the learning of latent user and recipe factors and the quality of recommendations, especially for new or less active users and items. Incorporating Maximal Marginal Relevance (MMR) as a post-processing step addressed the challenge of limited diversity, resulting in more varied and serendipitous recommendation lists that balance user preferences with novel suggestions. This diversification comes at the cost of minor losses in accuracy metrics, but significantly enhances the discovery aspect crucial in culinary applications.

Our results indicate that while traditional collaborative filtering methods remain powerful for recipes, real-world recommender systems should combine accuracy with meaningful diversity, especially in domains characterized by subjective tastes and a multitude of options. Properly integrating diversification algorithms such as MMR can improve user satisfaction and engagement, opening the door to smarter, more enjoyable applications—be it cooking assistants, nutrition platforms, or cultural culinary explorers.

Future work may explore hybrid models that leverage both collaborative and content-based signals (e.g., ingredient lists, cuisine styles) and employ more advanced diversification strategies or fairness constraints. Additionally, improvements in cold-start handling and adaptation to dynamic user preferences (e.g., seasonal or dietary changes) could further enhance the user experience and practical impact of recipe recommenders.